An edge between consecutive vertex classes can be accepted while enough pebbles remain

Signe Lundqvist, Tovohery Randrianarisoa, Klara Stokes, Joannes Vermant

arxiv:2306.05050v3; Creative Commons Attribution 4.0 International

Editorial note. The counted statement is the author's labelled lemma environment at source0.tex lines 913-917; its proof environment opens at line 918, immediately after \end{lemma}, so the pairing is by adjacency and the retained proof range 918-929 is the whole environment.

Editorial note. Two statements are retained as context: the numbered system of inequalities at source lines 241-245, which the counted statement assumes, and the invariants lemma at source lines 877-885, whose invariant the proof uses at source line 920. These are the only outside labels of the retained material.

Editorial note. The retained proof carries no citation, so no bibliography is needed; the source ships both a bibliography.bib and a prepared retained bibliography for the cards of this source that do cite.

Editorial note. Preamble: the author's own package lines, theorem-environment family and macro block (source lines 4-39); the drawing and algorithm packages of the source are not retained because the retained material does not use them.

Editorial note. Metadata provenance: the licence evidence file records source_version 2306.05050v3 with status supported and the card uses that value; the author list is transcribed from the authors_index key of the source record file of the pinned copy, and the title from its title key.

Editorial note. The source carries no label-number dump and no counter values are imposed, so the printed numbering is the wrapper's default; the identity of the counted item is its author label.

Editorial scope. One original result of the pinned author manuscript, counted once: the step of the pebble game accepting an edge between consecutive vertex classes (author label lemma-add-edge(poset)).

Allowed standard prerequisites (editorial).

- The pebble game on a poset. An algorithm that keeps a number of pebbles on each vertex and accepts edges while respecting the sparsity parameters, with a reachability operation used to move pebbles along accepted edges.. It is the algorithm whose behaviour the statement describes.
- Pebble, span and out counts. For a subset of vertices the number of pebbles, the span and the out-degree are related to the weighted size of the classes the subset meets by the algorithm's invariants.. They are the quantities used to count pebbles inside a reachable set.

- Reachable set of a vertex. The set of vertices that can be reached from a given vertex by following accepted edges, respecting the direction conventions of the game.. It is the set on which the counting argument is performed.

$$k_i + k_{i+1} > l_{i,i+1}, \tag{1}$$
$$k_{i+1} + l_{i-1,i} > l_{i,i+1}, \tag{2}$$
$$l_{1,2} \leq \ldots \leq l_{n-1,n}. \tag{3}$$

Lemma 1. *Let $K \in \mathbb{Z}^n_{>0}, L \in \mathbb{Z}^{n-1}_{\geq 0}$ be such that inequalities (1), (2) and (3) are valid. Let $G = (V, E)$ be a graph such that the vertex set has a partition $V = V_1 \cup \cdots \cup V_n$ such that all edges $e \in E$ have endpoints in $V_i \cup V_{i+1}$ for some $1 \leq i < n$. The following are invariants of the algorithm*

1. *For all $v \in V_i$, one has $\mathrm{peb}(v) + \mathrm{out}(v) = k_i$.*
2. *Suppose that $V' \subseteq V$. Then*
$$\mathrm{peb}(V') + \mathrm{span}(V') + \mathrm{out}(V') = \sum_{i=1}^{n} k_i |V' \cap V_i|.$$
3. *Consider a nonempty subset $V' \subseteq V$. Then*
$$\mathrm{peb}(V') + \mathrm{out}(V') \geq l(V').$$
4. *Consider a subset $V' \subseteq V$. Then*
$$\mathrm{span}(V') \leq \sum_{i=1}^{n} k_i |V' \cap V_i| - l(V').$$

Lemma 2. *Let $K \in \mathbb{Z}^n_{>0}, L \in \mathbb{Z}^{n-1}_{\geq 0}$ be such that inequalities (1), (2) and (3) are valid. Let $G = (V, E)$ be a graph such that the vertex set has a partition $V = V_1 \cup \cdots \cup V_n$ such that all edges $e \in E$ have endpoints in $V_i \cup V_{i+1}$ for some $1 \leq i < n$.*

Suppose that after running some steps of the algorithm D is the set of edges we accepted, where all edges in D are between V_a and V_{a+1} for $a \leq i$. Suppose that e is an edge of G which has not been accepted yet such that $e = (uv)$, where $u \in V_i$ and $v \in V_{i+1}$. Then the edge can be accepted (after perhaps moving some pebbles).

Proof. If there are more than $l_{i,i+1}$ pebbles at u and v together, we can add the edge and remove a pebble. So, we may assume that $\mathrm{peb}(u) + \mathrm{peb}(v) < l_{i,i+1} + 1$. Let $V' = \mathrm{Reach}(u) \cup \mathrm{Reach}(v)$, and let $E_G(V')$ denote the edges induced by V' in G. Note that $V' \cap V_i$ and $V' \cap V_{i+1}$ are non-empty since $u \in V' \cap V_i$ and $v \in V' \cap V_{i+1}$. Furthermore $V' \cap V_k = \emptyset$, for $k \geq i + 2$, since these edges get processed at some later point. Since G is sparse,

$$\mathrm{span}(V') < |E_G(V')| \leq \sum_{j=1}^{i+1} k_j |V' \cap V_j| - l_{i,i+1},$$

where the first inequality is strict since $e \in E_G(V')\backslash\text{span}(V')$, and $\max\{l_{k,k+1}\} = l_{i,i+1}$ since the $l_{k,k+1}$ are increasing. By the definition of Reach, $\text{out}(V') = 0$ and by invariant 2 in Lemma 1,

$$\text{span}(V') + \text{peb}(V') = \sum_{j=1}^{i+1} k_j |V_j \cap V'|$$

and thus

$$\text{peb}(V') > l_{i,i+1}.$$

Hence, there is a vertex $w \in V'$ such that $w \notin \{v, u\}$ with at least one pebble left. Since it is reachable by either v or u, we can move that pebble (changing the direction of the path). Repeat this process until we get enough pebbles together at u and v so that we can accept the edge (uv) and add it to D. $\square$